

CSCE 637 COMPLEXITY THEORY · MODULE 13 · LECTURE NOTES

What Complexity Tells You

The honest summary of a field with an open centre.

MODULE OUTCOMES

- State what is proved and what is conjectured.
- Avoid the standard overreaches.
- Use a hardness result productively.
- Place this course relative to CSCE 627 and CSCE 658.
- State what a complexity claim can honestly assert.

1 What is proved and what is conjectured

Statement	Status
$P \neq \text{EXPTIME}$, $NL \neq \text{PSPACE}$, and the hierarchy theorems generally	PROVED (Module 07 §1)
Cook–Levin, and the NP-completeness of thousands of problems	PROVED (Module 04)
$\text{PSPACE} = \text{NPSPACE}$ (Savitch); $NL = \text{co-NL}$ (Immerman–Szelepcsényi)	PROVED (Module 06 §2)
$IP = \text{PSPACE}$; the PCP theorem; zero-knowledge for all of NP	PROVED (Module 10)
Three barrier results: relativisation, natural proofs, algebrisation	PROVED (Modules 07, 09)
$P \neq NP$	OPEN. Widely believed (Module 03 §3)
$NP \neq \text{co-NP}$; the polynomial hierarchy being strict; $NC \neq P$; $BPP = P$	ALL OPEN
Any super-linear lower bound on general circuits for an explicit function	OPEN. The best known is about $5n$ (Module 09 §2)

A great deal is proved and the central question is not — and the structural observation worth making is that the proved results are mostly *equalities* and *completeness* results, while almost every *separation* is open. Which is exactly what the barrier results explain (Module 07 §4): the field's techniques are good at showing two things are the same and bad at showing they differ, and that is now a theorem rather than an observation.

2 The overreaches

The claim	The correction
'NP-complete means intractable.'	SAT solvers handle instances with millions of variables (Module 04 §4, CSCE 625 Module 08). The label is about worst-case asymptotics, not about your instances.
'NP-hard means it takes exponential time.'	It means no polynomial algorithm is known and finding one would resolve P versus NP — which is conditional (Module 03 §2). No exponential lower bound is proved for any NP-complete problem.
'Quantum computers will solve NP-complete problems.'	BQP is not believed to contain NP , and Grover's algorithm gives a quadratic speedup rather than an exponential one (CSCE 627 Module 11 §3, CSCE 640).
'Polynomial means fast.'	n^5 with a large constant is neither fast nor practical (Module 01 §4, Module 02 §1's objections).
' $P \neq NP$ is basically proved.'	The best general circuit lower bound is about $5n$ and we believe the truth is exponential (Module 09 §2). The gap is the whole problem.
'This problem is NP-complete, so give up.'	Five productive responses, all routine (Module 04 §4, §3 below). The most consequential overreach, because it stops work that would have succeeded.

Every overreach treats a conditional, worst-case, asymptotic statement as an unconditional, typical-case, practical one. Which is the same shape as CSCE 627 Module 13 §2's errors, with conditionality added as a fourth way to go wrong — and recognising the shape is faster than refuting each instance.

3 Using a hardness result productively

- **Run a solver first. It costs an afternoon and it frequently just works** (CSCE 669 Module 13 §1) — and discovering that your instances are easy is the cheapest possible outcome.
- **Find the parameter** (Module 12 §1), **and measure whether it is actually small in your data. The highest-value step in the list**, and the one most often skipped because parameterised complexity is less widely taught.
- **Look up both approximation sides** (Module 11 §4), **and implement the known optimal algorithm if the question is closed** — twenty minutes, and it sometimes ends the project in the best way.
- **Restrict the problem** — planar, bounded degree, bipartite, interval, chordal. **Frequently your instances already satisfy a restriction that makes the problem polynomial**, and nobody checked.
- **And always compute a bound**, even when using a heuristic, **so the result becomes 'within 7% of optimal' rather than a number** (CSCE 669 Module 13 §3). **All five are routine, and the hardness result is what tells you to do them rather than continue looking for an exact polynomial algorithm** — which is the classification's whole operational value (Module 02 §4).

4 The semester, and the honest claim

Course	Central claim form	Certainty
CSCE 627	'No algorithm exists.'	Unconditional. Permanent.
CSCE 637 (this one)	'No polynomial-time algorithm, unless $P = NP$.'	Conditional on a question open for fifty years , with three theorems explaining why it is open.
CSCE 658	'Randomness helps here.'	Proved in specific cases; whether it is ever essential is open (Module 08 §3's derandomisation question).

The certainty decreases as the resource under study becomes more refined — computability is settled, time complexity is mostly conjectural, and the value of randomness is open in principle. **Which is the honest shape of a theory semester, and is worth stating rather than presenting the three as equivalent:** a student who leaves believing $P \neq NP$ is proved has been badly served.

- **'This problem is NP-complete; here is the reduction from 3-SAT and here is the certificate.'** Checkable by anyone, and both halves present (Module 04 §1).
- **'No polynomial-time algorithm exists unless $P = NP$ ' — with the conditionality stated, because it is real** and because omitting it is the overreach of §2's second row.
- **'The best achievable approximation ratio is $7/8$, and $7/8$ is optimal unless $P = NP$.'** Both sides, with the condition attached (Module 11 §1).
- **'The problem is fixed-parameter tractable in treewidth, and our instances have treewidth under 12.'** A theorem plus a measurement, which is the strongest useful form available — it is rigorous and it is about your actual problem, which no other claim form in this course manages simultaneously.

- **And what you cannot honestly say: 'this is intractable'** — unqualified, unconditional, and implicitly about your instances. **Three errors in one word**, and it is the most common thing said about an NP-complete problem.

WHERE THIS COURSE LEAVES YOU

You can prove a problem NP-complete with both halves present, place problems in co-NP, PSPACE, and the polynomial hierarchy by counting quantifiers, and prove a hierarchy theorem from scratch.

You know why the central question is open — three barrier results — and what the field can and cannot prove, including the $5n$ figure, which is the single most calibrating number in the subject.

And you can take a hard problem and do something productive with it: find a parameter, build a kernel, look up both approximation bounds, check a fine-grained lower bound. **Which is the practical skill, and it is Module 12's material rather than Module 04's.**

The closing rule is the program's, unchanged across twenty-three courses: state what you measured, state what you assumed, and never claim more than you established. In complexity theory it means stating the conditionality — because almost everything in this course rests on a conjecture, and saying so is the difference between a result and a slogan.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aaronson — $P = ? NP$ (free survey)	§1 and §2, done properly by someone in the field — and the best single thing to read after this course.
Arora & Barak — the whole book, reread (free draft)	Free, and the second reading is different from the first — particularly chapters 23 and 20.
Cygan et al. — Parameterized Algorithms (free PDF)	If one book from this course should stay on your desk, it is this one — Module 12 is where the practical value is.
Garey & Johnson, appendix; and the modern successors	The reduction library , which is what you will actually use — keep a copy or a bookmark.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write down which results from this course are proved and which are conjectured**, from memory.

- 2 **Explain why the proved results are mostly equalities**, in terms of the barriers.
- 3 **Find three of §2's overreaches in the wild** and write the correction for each.
- 4 **Explain the common shape of all six** in one sentence.
- 5 **Take an NP-hard problem from your own work and apply all five of §3's steps**. Report which helped.
- 6 **Compare the three Semester 8 courses' claim forms** and say which you trust most.
- 7 **Write the honest claim** for your Project 1 NP-completeness result.
- 8 **Write the honest claim** for your Project 2 problem, in §4's fourth form.
- 9 **Revisit what you wrote in Module 01's last exercise** and report what changed.
- 10 **Project 2 is now due**. Submit the problem with its hardness established, the solver results, the easy and hard instance families, the parameter assessment, the approximation picture with both sides, any fine-grained bound, and a written answer to 'what does NP-hardness actually predict here?'

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five proved results and four open ones.
- 2 Why are the proved results mostly equalities?
- 3 Give six overreaches and the correction to each.
- 4 What shape do all six share?
- 5 Give the five productive responses to a hardness result.
- 6 Which is the highest-value step, and why is it most often skipped?
- 7 Compare the three theory courses' claim forms and certainties.
- 8 Give four honest claim forms and the one you cannot use.
- 9 Which claim form is strongest, and why?
- 10 In this course, what does 'state what you assumed' mean?